SISTEMAS OPERATIVOS 2026/2027 — TRABALHO PRÁTICO 2

Relatório do Simulador de Memória Virtual

Avaliação Comparativa dos Algoritmos de Substituição de Páginas

Autores: [Nome do Estudante / Grupo]

Repositório GitHub: https://github.com/alexandrelau-lei/SO2026-T2-codigo-base.git

Algoritmos VM: VM_FIFO, VM_RANDOM, VM_LRU, VM_NRU, VM_CLOCK

Escalonador: FIFO / RR (Round-Robin)

1. Introdução e Enquadramento

Este relatório descreve a implementação e os testes dos algoritmos de substituição de páginas desenvolvidos para o simulador `ossim`. O objetivo principal consistiu em simular a gestão de memória virtual sob diferentes estratégias de substituição de molduras e analisar as respetivas métricas de faltas de página (*Page Faults*) e acessos a disco (*Swap Reads* e *Swap Writes*).

2. Algoritmos Implementados

Algoritmo	Mecanismo Principal	Critério de Seleção de Vítima
VM_FIFO	Ponteiro circular (`fifo_hand`)	Substitui a página carregada há mais tempo na memória física.
VM_RANDOM	`rand() % num_frames`	Seleciona uma moldura válida de forma totalmente aleatória.
VM_LRU	Scan de `last_accessed`	Substitui a página com o menor timestamp de acesso (menos recentemente usada).
VM_NRU	Classificação por bits (R, D)	Categoriza em 4 classes: 0 (R=0, D=0), 1 (R=0, D=1), 2 (R=1, D=0) e 3 (R=1, D=1). Evicta da menor classe disponível.
VM_CLOCK	Ponteiro circular (`clock_hand`)	Dá segunda oportunidade a páginas com R=1 (passa R a 0) e avança até encontrar a primeira com R=0.

3. Instrumentação e Registo de Evicções

Para comprovar a exatidão das evicções e da manipulação dos bits de estado, acrescentaram-se logs detalhados no simulador:

```
[FAULT] Processo 1 -> Pagina Virtual 4. Substitui Frame 1 (Pagina Anterior 2) [NRU Detail] Selecionada moldura da Classe 2 (R=1, D=0) [EVICTION] Moldura 1 (Página 2, Processo 1) estava LIMPA (D=0) -> Sem Swap Write [FAULT] Processo 1 -> Pagina Virtual 6. Substitui Frame 0 (Pagina Anterior 1) [NRU Detail] Selecionada moldura da Classe 3 (R=1, D=1) [EVICTION] Moldura 0 (Página 1, Processo 1) estava DIRTY (D=1) -> Gerou Swap Write
```

4. Tabela de Resultados Experimentais

Cenário 1: `basico.csv`

Algoritmo	Molduras	Page Faults	Swap Read	Swap Write	Total I/O
FIFO	3	4	4	0	**4**
RANDOM	3	4	4	0	**4**
LRU	3	4	4	0	**4**
NRU	3	4	4	0	**4**
CLOCK	3	4	4	0	**4**
LRU	2	5	5	0	**5**
LRU	4	4	4	0	**4**

Cenário 2: `nru_dirty.csv` (Testes de Escrita / Dirty Bit)

Algoritmo	Molduras	Page Faults	Swap Read	Swap Write	Total I/O
FIFO	3	7	7	1	**8**
LRU	3	6	6	1	**7**
NRU	3	7	7	1	**8**
CLOCK	3	6	6	1	**7**

5. Análise dos Resultados e Conclusão

1. **Capacidade da Memória Física:** No teste do LRU com `basico.csv`, a redução para 2 molduras causou a evicção prematura da página 1, aumentando o total de faltas de página para 5. Com 4 molduras, o simulador obteve um `[HIT]` para a página 1, reduzindo as faltas de página para o mínimo possível (4).
2. **Desempenho LRU / CLOCK vs. FIFO:** No cenário `nru_dirty.csv`, o LRU e o CLOCK obtiveram 6 faltas de página contra 7 do FIFO, demonstrando maior eficiência ao tirar proveito da localidade temporal.
3. **Gestão de Páginas Modificadas:** Todos os algoritmos efetuaram 1 operação de *Swap Write* ao evictar uma página com bit dirty ativo ($D=1$), garantindo que os dados alterados foram salvaguardados em disco antes da substituição.